

Computer Networks Lab

Assignment 3

Network Simulation using NS-3

Reliable Data Transfer & Distance Vector Routing

CS342

July–November 2025

Group Number:	04
Member 1:	Kavya Kumar Agrawal (230101053)
Member 2:	Pratyush Kumar Swain (230101079)
Member 3:	Ritvij Gopal (230101085)
Member 4:	Parv Goyal (230101124)
Submission Date:	October 17, 2025

Department of Computer Science and Engineering
Indian Institute of Technology

Contents

1	Problem 1: Reliable Data Transfer using NS-3	3
1.1	Introduction	3
1.2	Theoretical Background	3
1.2.1	Principles of Reliable Data Transfer	3
1.2.2	Alternating Bit Protocol (Stop-and-Wait)	4
1.2.3	Go-Back-N Protocol	4
1.3	Finite State Machines	4
1.3.1	ABP Sender FSM	5
1.3.2	ABP Receiver FSM	5
1.3.3	GBN Receiver FSM	6
1.3.4	GBN sender FSM	6
1.4	Implementation Details	7
1.4.1	NS-3 Simulation Setup	7
1.4.2	Part A: Alternating Bit Protocol Implementation	7
1.4.3	Part B: Go-Back-N Protocol Implementation	10
1.5	Experimental Setup and Methodology	11
1.5.1	Experiment Design	12
1.5.2	Performance Metrics Collection	12
1.6	Results and Analysis	13
1.6.1	ABP Results	13
1.6.2	GBN Results	14
2	Problem 2: TCP Congestion Control	17
2.1	Introduction	17
2.2	Theoretical Background	17
2.2.1	TCP Congestion Control Fundamentals (Problem 2)	18
2.2.2	TCP Variants Overview (Problem 2)	18
2.3	Network Topology and Setup	18
2.3.1	Problem 2: TCP Topology	19
2.4	Implementation	19
2.5	Results and Analysis	19
2.5.1	Problem 2: Task 1 - Congestion Window Dynamics	19
2.5.2	Problem 2: Task 2 - Cumulative Packet Drops	22
2.5.3	Problem 2: Task 3 - Cumulative Bytes Transferred	24
3	Problem 3: Distance Vector Routing Algorithm and Count-to-Infinity Problem	26
3.1	Introduction	26
3.2	Theoretical Background	27
3.2.1	Bellman-Ford Equation	27
3.2.2	Algorithm Operation	27
3.2.3	Count-to-Infinity Problem	27
3.3	Network Topology	27
3.4	Mitigation Techniques	27
3.4.1	Split Horizon	28
3.4.2	Poison Reverse	28
3.5	Implementation Code	29
3.6	Simulation Results	31

3.6.1	Initial Convergence (Before Link Failure)	31
3.6.2	Mitigation Results: Split Horizon	33
3.6.3	Mitigation Results: Poison Reverse	34

1 Problem 1: Reliable Data Transfer using NS-3

1.1 Introduction

Reliable data transfer is a fundamental requirement in computer networks to ensure that data transmitted from a sender reaches the receiver correctly and in order, despite the unreliable nature of the underlying network infrastructure. Network channels may lose, corrupt, reorder, or duplicate packets due to various factors such as buffer overflows, transmission errors, routing changes, and hardware failures.

This problem focuses on implementing and analyzing two classical reliable data transfer protocols: the Alternating Bit Protocol (Stop-and-Wait) and the Go-Back-N protocol. Both protocols use sequence numbers, acknowledgments, checksums, and timeouts to achieve reliability, but differ significantly in their efficiency and complexity.

1.2 Theoretical Background

Theory

1.2.1 Principles of Reliable Data Transfer

Reliable data transfer protocols must address three main challenges:

1. **Bit Errors:** Data corruption during transmission due to noise, interference, or signal degradation. Detected using checksums or cyclic redundancy checks (CRC).
2. **Packet Loss:** Complete loss of packets in the network due to buffer overflow at routers, link failures, or routing loops. Detected using timeouts.
3. **Packet Reordering:** Out-of-order delivery due to different paths taken through the network. Handled using sequence numbers.

Key Mechanisms:

- **Sequence Numbers:** Each packet is assigned a unique sequence number to distinguish new data from retransmissions and to detect duplicates.
- **Acknowledgments (ACKs):** Receiver sends feedback to sender confirming successful receipt of packets.
- **Negative Acknowledgments (NAKs):** Receiver explicitly notifies sender about corrupted or missing packets.
- **Checksums:** Mathematical function computed over packet data to detect bit errors.
- **Timeouts:** Sender maintains timers; if ACK not received within timeout period, packet is retransmitted.
- **Retransmissions:** Lost or corrupted packets are transmitted again.

1.2.2 Alternating Bit Protocol (Stop-and-Wait)

Theory

The Alternating Bit Protocol (ABP), also known as Stop-and-Wait ARQ (Automatic Repeat Request), is the simplest reliable data transfer protocol. It uses a single-bit sequence number that alternates between 0 and 1.

Operation:

1. Sender transmits packet with sequence number (0 or 1)
2. Sender waits for acknowledgment before sending next packet
3. Receiver sends ACK with expected sequence number
4. If timeout occurs or NAK received, sender retransmits
5. Upon receiving correct ACK, sender alternates sequence number and sends next packet

Advantages:

- Simple implementation
- Minimal buffer requirements
- Easy error detection and recovery

Disadvantages:

- Poor channel utilization
- Low throughput, especially on high bandwidth-delay product networks
- Sender idle while waiting for acknowledgment

1.2.3 Go-Back-N Protocol

Theory

Go-Back-N (GBN) is a sliding window protocol that allows the sender to transmit multiple packets without waiting for individual acknowledgments, improving throughput significantly compared to Stop-and-Wait.

Key Concepts:

- **Window Size (N):** Maximum number of unacknowledged packets sender can have in pipeline
- **Base:** Sequence number of oldest unacknowledged packet
- **Nextseqnum:** Sequence number of next packet to be sent
- **Cumulative ACK:** Receiver acknowledges all packets up to and including sequence number n

Operation:

1. Sender can transmit up to N packets without waiting for ACKs
2. Single timer maintained for oldest unacknowledged packet
3. Receiver sends cumulative ACK for last correctly received in-order packet
4. On timeout or receiving NAK, sender retransmits all packets from base to nextseqnum-1
5. Receiver discards out-of-order packets (simplifies receiver)

Advantages:

- Higher throughput than Stop-and-Wait
- Better utilization of network bandwidth

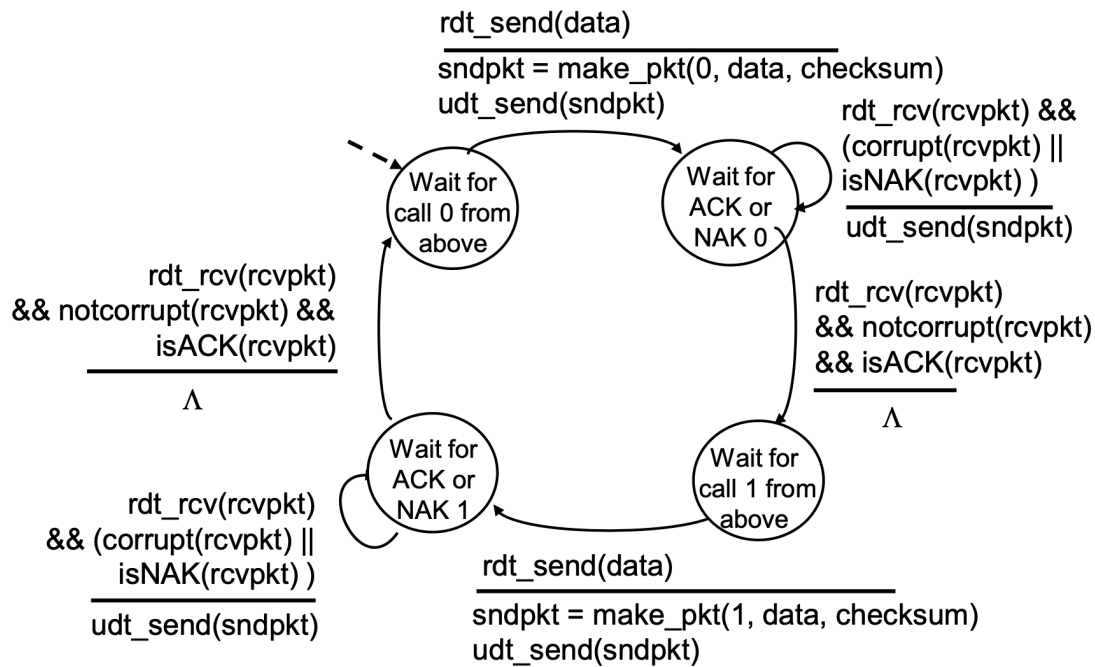
Disadvantages:

- Retransmits correctly received packets after lost packet
- Requires larger sequence number space

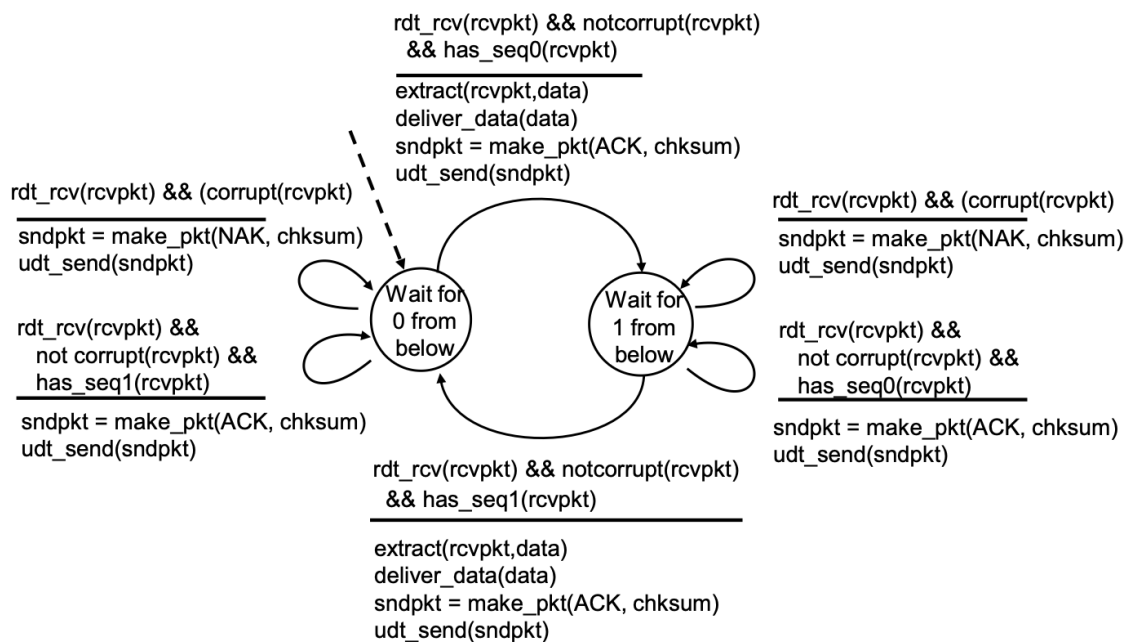
1.3 Finite State Machines

1.3.1 ABP Sender FSM

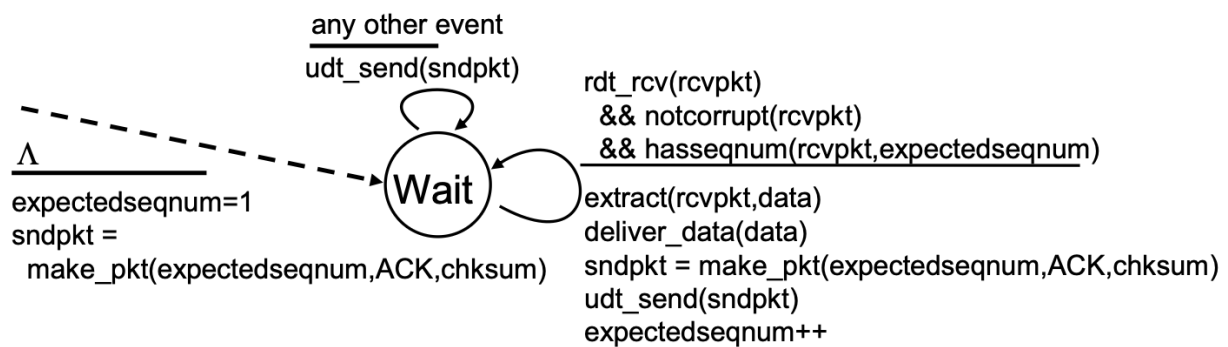
The sender in Alternating Bit Protocol operates in two states corresponding to the two sequence numbers (0 and 1).



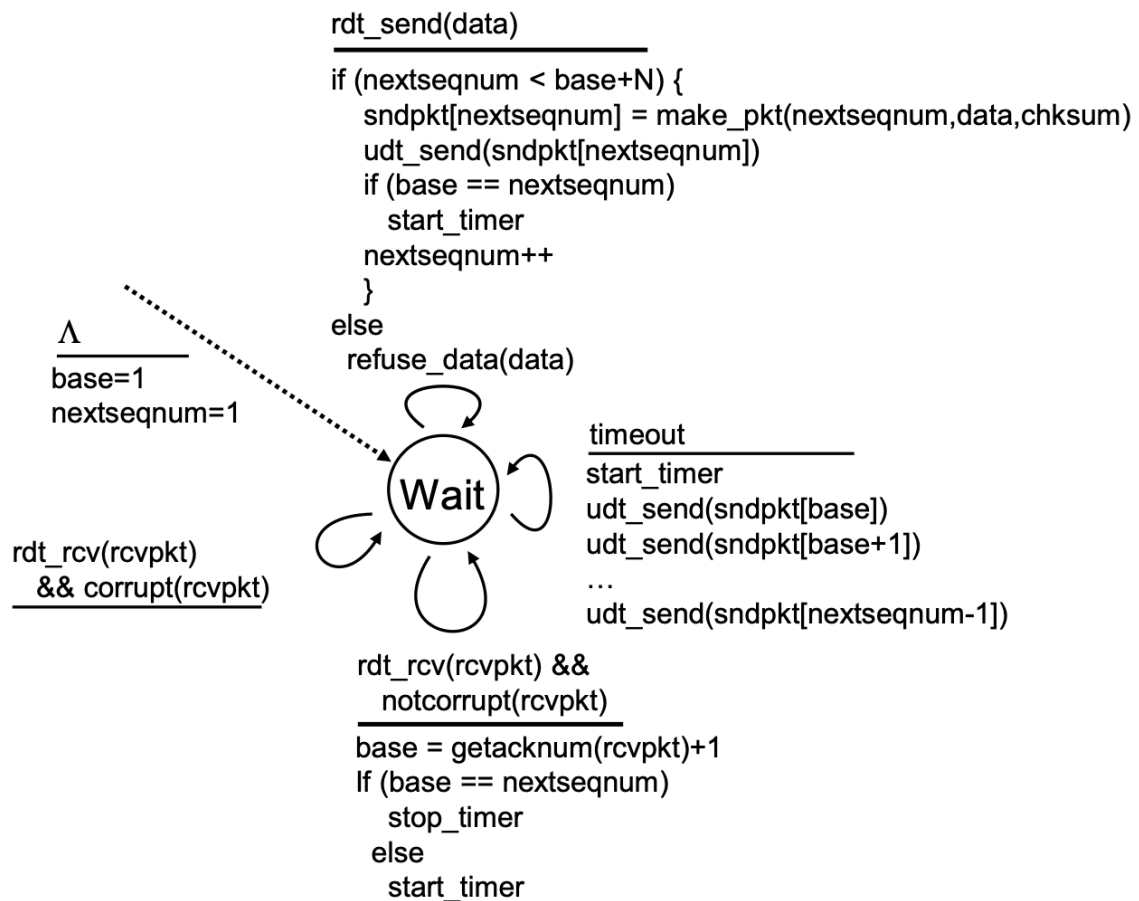
1.3.2 ABP Receiver FSM



1.3.3 GBN Receiver FSM



1.3.4 GBN sender FSM



1.4 Implementation Details

1.4.1 NS-3 Simulation Setup

Theory

Network Topology:

- Two nodes (Node 0 and Node 1) connected via point-to-point link
- Link bandwidth: 1 Mbps
- Link delay: 10 ms
- Queue type: DropTail

Error Model Configuration:

The RateErrorModel in NS-3 is used to introduce artificial packet loss and corruption:

- Packet Error Rate (PER): Configurable (e.g., 0.01, 0.05, 0.1)
- Error Unit: Packet-based
- Random Variable: Uniform distribution

Simulation Parameters:

- Simulation time: 20 seconds
- Packet size: 1024 bytes
- Timeout value: $2 \times \text{RTT}$ (adjusted based on link parameters)

1.4.2 Part A: Alternating Bit Protocol Implementation

Implementation Overview:

The ABP implementation consists of custom sender and receiver applications in NS-3. The sender application maintains state for the current sequence number and handles timeouts, while the receiver validates packets and sends appropriate acknowledgments.

```
void SenderSimulator::TransmitPacket()
{
    if (m_packetsSent < m_totalPackets && !m_waitingAck)
    {
        ABPPacket pkt(m_currentSeq, 0, 0);
        Ptr<Packet> packet = Create<Packet>((uint8_t *)&pkt, sizeof(pkt));
        m_socket->Send(packet);

        NS_LOG_UNCOND("\nSender: Sent packet Seq=" << (int)m_currentSeq
                      << " at " << Simulator::Now().GetSeconds() << "s");

        m_waitingAck = true;
        m_lastAckReceived = false;
        if (m_timeoutEvent.IsRunning())
            Simulator::Cancel(m_timeoutEvent);

        m_timeoutEvent = Simulator::Schedule(Seconds(2.0), &SenderSimulator::OnTimeout, this);
    }
}
```



```

void SenderSimulator::HandleIncomingPacket(Ptr<Socket> sock)
{
    Ptr<Packet> packet = sock->Recv();
    ABPPacket pkt;
    packet->CopyData((uint8_t *)&pkt, sizeof(pkt));

    if (pkt.nakFlag)
    {
        if (pkt.acknowledgement != m_currentSeq) return;

        NS_LOG_UNCOND("Sender: Received NAK for Seq=" << (int)pkt.acknowledgement
            << " at " << Simulator::Now().GetSeconds() << "s. Retransmitting...");
        m_waitingAck = false;
        m_retransmitCount++;
        TransmitPacket();
        return;
    }

    if (pkt.acknowledgement == m_currentSeq)
    {
        Simulator::Cancel(m_timeoutEvent);
        m_packetsSent++;
        m_currentSeq = (m_currentSeq + 1) % 2;
        m_waitingAck = false;
        m_lastAckReceived = true;

        NS_LOG_UNCOND("Sender: Received ACK for Seq=" << (int)pkt.acknowledgement
            << " at " << Simulator::Now().GetSeconds());

        if (m_metricsLogger)
            m_metricsLogger->OnPacketAck();

        if (m_packetsSent < m_totalPackets)
            Simulator::Schedule(Seconds(0.0), &SenderSimulator::TransmitPacket, this);
        else
            NS_LOG_UNCOND("\nSender: All packets successfully transmitted.");
    }
}

void SenderSimulator::OnTimeout()
{
    if (m_lastAckReceived) return;

    NS_LOG_UNCOND("Sender: Timeout for Seq=" << (int)m_currentSeq
        << " at " << Simulator::Now().GetSeconds() << "s. Retransmitting...");
    m_waitingAck = false;
    m_retransmitCount++;
    TransmitPacket();
}

```

```

void ReceiverSimulator::OnPacketReceived(Ptr<Socket> sock)
{
    Ptr<Packet> packet;
    Address senderAddr;

    while ((packet = sock->RecvFrom(senderAddr)))
    {
        bool isCorrupt = m_errModel->IsCorrupt(packet);

        ABPPacket pkt;
        packet->CopyData((uint8_t *)&pkt, sizeof(pkt));

        if (isCorrupt)
        {
            NS_LOG_UNCOND("Receiver: Packet corrupted! Sending NAK for Seq=" << (int)m_expectedSequence
                          << " at " << Simulator::Now().GetSeconds() << "s");
            ABPPacket nakPkt(0, m_expectedSequence, 1);
            Ptr<Packet> nakPacket = Create<Packet>((uint8_t *)&nakPkt, sizeof(nakPkt));
            sock->SendTo(nakPacket, 0, senderAddr);
            continue;
        }

        if (pkt.sequence == m_expectedSequence)
        {
            NS_LOG_UNCOND("Receiver: Received new packet Seq=" << (int)pkt.sequence
                          << " at " << Simulator::Now().GetSeconds() << "s");
            m_expectedSequence = (m_expectedSequence + 1) % 2;

            ABPPacket ackPkt(0, pkt.sequence, 0);
            Ptr<Packet> ackPacket = Create<Packet>((uint8_t *)&ackPkt, sizeof(ackPkt));
            sock->SendTo(ackPacket, 0, senderAddr);
            NS_LOG_UNCOND("Receiver: Sent ACK=" << (int)ackPkt.acknowledgement
                          << " at " << Simulator::Now().GetSeconds() << "s");
            m_sentDuplicateNak = false;
        }
        else
        {
            if (!m_sentDuplicateNak)
            {
                NS_LOG_UNCOND("Receiver: Duplicate/out-of-order packet Seq=" << (int)pkt.sequence
                              << ". Sending NAK for Seq=" << (int)m_expectedSequence
                              << " at " << Simulator::Now().GetSeconds() << "s");
                ABPPacket nakPkt(0, m_expectedSequence, 1);
                Ptr<Packet> nakPacket = Create<Packet>((uint8_t *)&nakPkt, sizeof(nakPkt));
                sock->SendTo(nakPacket, 0, senderAddr);
                m_sentDuplicateNak = true;
            }
        }
    }
}

```

```

// ===== ABP Packet Header =====
struct ABPPacket
{
    uint8_t sequence;
    uint8_t acknowledgement;
    uint8_t nakFlag;      // 0 = ACK, 1 = NAK
    uint16_t checksum;

    ABPPacket(uint8_t seq = 0, uint8_t ack = 0, uint8_t nak = 0)
        : sequence(seq), acknowledgement(ack), nakFlag(nak)
    {
        checksum = sequence + acknowledgement + nakFlag;
    }
};

```

seqnum	acknum	isNak	checksum
uint8_t	uint8_t	uint8_t	uint16_t

```
// ----- Packet Loss Model -----
Ptr<RateErrorModel> lossModel = CreateObject<RateErrorModel>();
lossModel->SetAttribute("ErrorRate", DoubleValue(0.1)); // 10% packet loss
lossModel->SetAttribute("ErrorUnit", StringValue("ERROR_UNIT_PACKET"));
devices.Get(1)->SetAttribute("ReceiveErrorModel", PointerValue(lossModel));

// ----- Packet Corruption Model -----
Ptr<RateErrorModel> corruptModel = CreateObject<RateErrorModel>();
Ptr<UniformRandomVariable> uniformVar = CreateObject<UniformRandomVariable>();
corruptModel->SetRandomVariable(uniformVar);
corruptModel->SetRate(0.01); // 1% corruption
```

1.4.3 Part B: Go-Back-N Protocol Implementation

Implementation Overview:

The GBN implementation extends the ABP design by introducing a sliding window mechanism at the sender. The sender maintains a buffer of unacknowledged packets and implements a single timer for the oldest unacknowledged packet.

```
// ----- Go-Back-N Packet Header -----
struct PacketHeaderGBN
{
    uint8_t seqNum;
    uint16_t checksum;

    PacketHeaderGBN(uint8_t num = 0) : seqNum(num) { checksum = seqNum; }
};
```

```

void GbnSender::SendWindowPackets()
{
    while (m_nextSeq < m_baseSeq + m_winSize && m_nextSeq < m_totalPkts)
    {
        DispatchPacket(m_nextSeq);
        if (m_baseSeq == m_nextSeq)
            m_timer = Simulator::Schedule(Seconds(2.0), &GbnSender::OnTimeout, this);
        ++m_nextSeq;
    }

    if (m_metrics)
        m_metrics->UpdateWindow(m_baseSeq, m_nextSeq, m_winSize);
}

void GbnSender::OnPacketReceived(Ptr<Socket> sock)
{
    Ptr<Packet> pkt = sock->Recv();
    PacketHeaderGBN hdr;
    pkt->CopyData((uint8_t *)&hdr, sizeof(hdr));

    uint8_t ackNum = hdr.seqNum % (SEQ_MAX + 1);
    NS_LOG_UNCOND("Sender: Received ACK for Seq=" << (int)ackNum << " at " << Simulator::Now().GetSeconds());

    while ((int)m_baseSeq % (SEQ_MAX + 1) != ((ackNum + 1) % (SEQ_MAX + 1)) && m_baseSeq < m_totalPkts)
    {
        if (m_metrics)
            m_metrics->OnPacketAked();
        ++m_baseSeq;
    }

    if (m_baseSeq == m_nextSeq)
        Simulator::Cancel(m_timer);
    else
    {
        Simulator::Cancel(m_timer);
        m_timer = Simulator::Schedule(Seconds(2.0), &GbnSender::OnTimeout, this);
    }

    SendWindowPackets();
}

void GbnSender::OnTimeout()
{
    NS_LOG_UNCOND("\nSender: Timeout! Retransmitting from Seq=" << m_baseSeq << " at " << Simulator::Now().GetSeconds() << "\n");
    if (m_metrics)
        m_metrics->OnRetransmission();

    for (uint32_t i = m_baseSeq; i < m_nextSeq; ++i)
        DispatchPacket(i);

    m_timer = Simulator::Schedule(Seconds(2.0), &GbnSender::OnTimeout, this);
}

```

```

void GbnReceiver::OnPacketReceived(Ptr<Socket> sock)
{
    Ptr<Packet> pkt;
    Address senderAddr;

    while ((pkt = sock->RecvFrom(senderAddr)))
    {
        bool isCorrupt = m_errorModel->IsCorrupt(pkt);

        PacketHeaderGBN hdr;
        pkt->CopyData((uint8_t *)&hdr, sizeof(hdr));

        uint8_t pktSeq = hdr.seqNum % (SEQ_MAX + 1);

        if (isCorrupt || pktSeq != (m_expectedSeq % (SEQ_MAX + 1)))
        {
            (const char [55])"\nReceiver: Packet lost/corrupted or out-of-order! Seq="
            NS_LOG_UNCOND("\nReceiver: Packet lost/corrupted or out-of-order! Seq=" << (int)pktSeq
                << ", Expected=" << (m_expectedSeq % (SEQ_MAX + 1))
                << " at " << Simulator::Now().GetSeconds());
            if (m_metrics)
                m_metrics->OnPacketLost();
        }

        if (pktSeq == (m_expectedSeq % (SEQ_MAX + 1)))
        {
            NS_LOG_UNCOND("Receiver: Received in-order packet Seq=" << (int)pktSeq
                << " at " << Simulator::Now().GetSeconds());
            m_expectedSeq = (m_expectedSeq + 1) % (SEQ_MAX + 1);
        }

        // Send cumulative ACK
        uint8_t ackNum = (m_expectedSeq - 1 + (SEQ_MAX + 1)) % (SEQ_MAX + 1);
        PacketHeaderGBN ackHdr(ackNum);
        Ptr<Packet> ackPkt = Create<Packet>((uint8_t *)&ackHdr, sizeof(ackHdr));
        sock->SendTo(ackPkt, 0, senderAddr);
        NS_LOG_UNCOND("Receiver: Sent cumulative ACK=" << (int)ackNum << " at " << Simulator::Now().GetSeconds());
    }
}

```

1.5 Experimental Setup and Methodology

1.5.1 Experiment Design

Multiple experiments were conducted to evaluate protocol performance under varying network conditions:

Experiment 1: Baseline Performance

- No packet loss or corruption
- Measure maximum achievable throughput
- Establish baseline for comparison

Experiment 2: Impact of Packet Loss

- Vary packet loss rate: 1%, 5%, 10%, 20%
- Measure retransmissions and throughput degradation
- Compare ABP vs. GBN performance

Experiment 3: Impact of Link Delay

- Vary link delay: 5 ms, 10 ms, 50 ms, 100 ms
- Analyze effect on protocol efficiency
- Evaluate window size optimization for GBN

Experiment 4: Window Size Analysis (GBN only)

- Test window sizes: $N = 2, 4, 8, 16$
- Identify optimal window size
- Analyze window utilization

1.5.2 Performance Metrics Collection

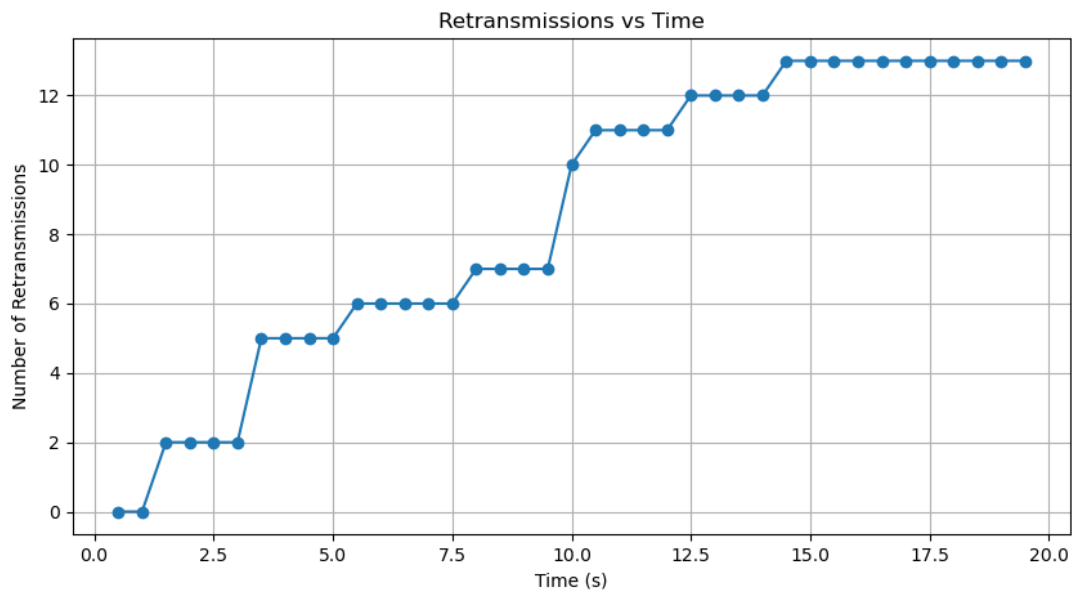
Using NS-3's Flow Monitor and custom packet tracing:

- **Throughput:** Total data successfully delivered per unit time (Mbps)
- **Goodput:** Application-level throughput excluding retransmissions
- **Retransmission Count:** Number of packets retransmitted
- **Packet Loss Rate:** Percentage of packets lost in network
- **End-to-End Delay:** Average time from transmission to successful reception
- **Window Utilization:** For GBN, percentage of window actively used

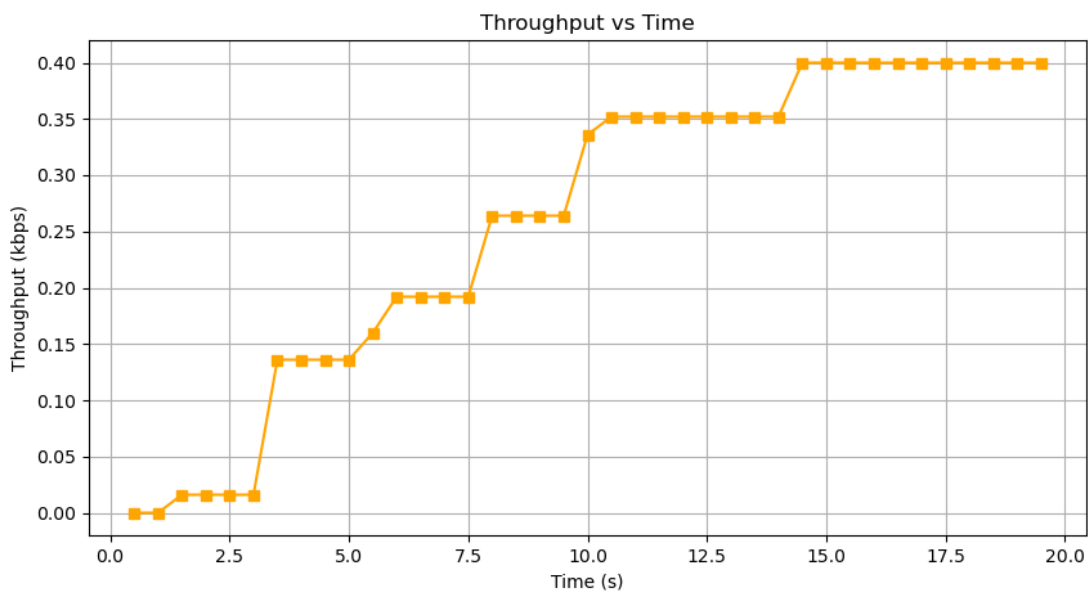
1.6 Results and Analysis

1.6.1 ABP Results

Retransmissions vs. Time:



Throughput vs. Time:



```
Sender: Sent packet Seq=1 at 10.0082s
Receiver: Received new packet Seq=1 at 10.0185s
Receiver: Sent ACK=1 at 10.0185s
Sender: Received ACK for Seq=1 at 10.0288

Sender: Sent packet Seq=0 at 10.0288s
Receiver: Packet corrupted! Sending NAK for Seq=0 at 10.0391s
Sender: Received NAK for Seq=0 at 10.0494s. Retransmitting...

Sender: Sent packet Seq=0 at 10.0494s
Sender: Timeout for Seq=0 at 12.0494s. Retransmitting...

Sender: Sent packet Seq=0 at 12.0494s
Sender: Timeout for Seq=0 at 14.0494s. Retransmitting...

Sender: Sent packet Seq=0 at 14.0494s
Receiver: Received new packet Seq=0 at 14.0597s
Receiver: Sent ACK=0 at 14.0597s
Sender: Received ACK for Seq=0 at 14.07

Sender: Sent packet Seq=1 at 14.07s
Receiver: Received new packet Seq=1 at 14.0802s
Receiver: Sent ACK=1 at 14.0802s
Sender: Received ACK for Seq=1 at 14.0905

Sender: Sent packet Seq=0 at 14.0905s
Receiver: Received new packet Seq=0 at 14.1008s
Receiver: Sent ACK=0 at 14.1008s
Sender: Received ACK for Seq=0 at 14.1111

Sender: Sent packet Seq=1 at 14.1111s
Receiver: Received new packet Seq=1 at 14.1214s
Receiver: Sent ACK=1 at 14.1214s
Sender: Received ACK for Seq=1 at 14.1317

Sender: Sent packet Seq=0 at 14.1317s
Receiver: Received new packet Seq=0 at 14.142s
Receiver: Sent ACK=0 at 14.142s
Sender: Received ACK for Seq=0 at 14.1523

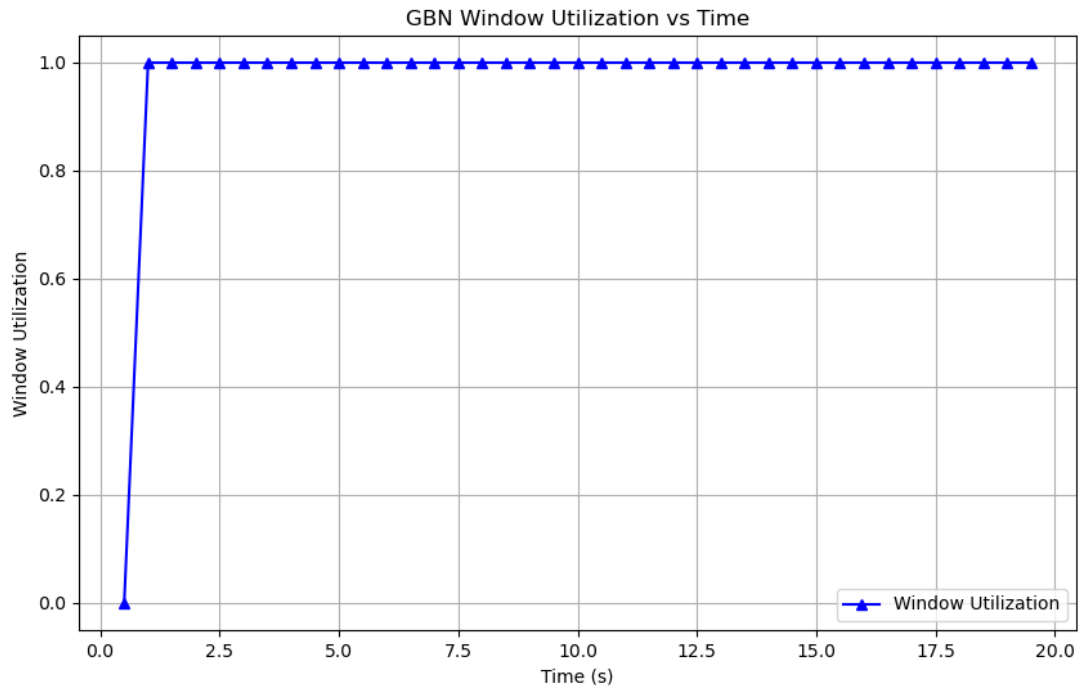
Sender: Sent packet Seq=1 at 14.1523s
Receiver: Received new packet Seq=1 at 14.1625s
Receiver: Sent ACK=1 at 14.1625s
Sender: Received ACK for Seq=1 at 14.1728

Sender: All packets successfully transmitted.

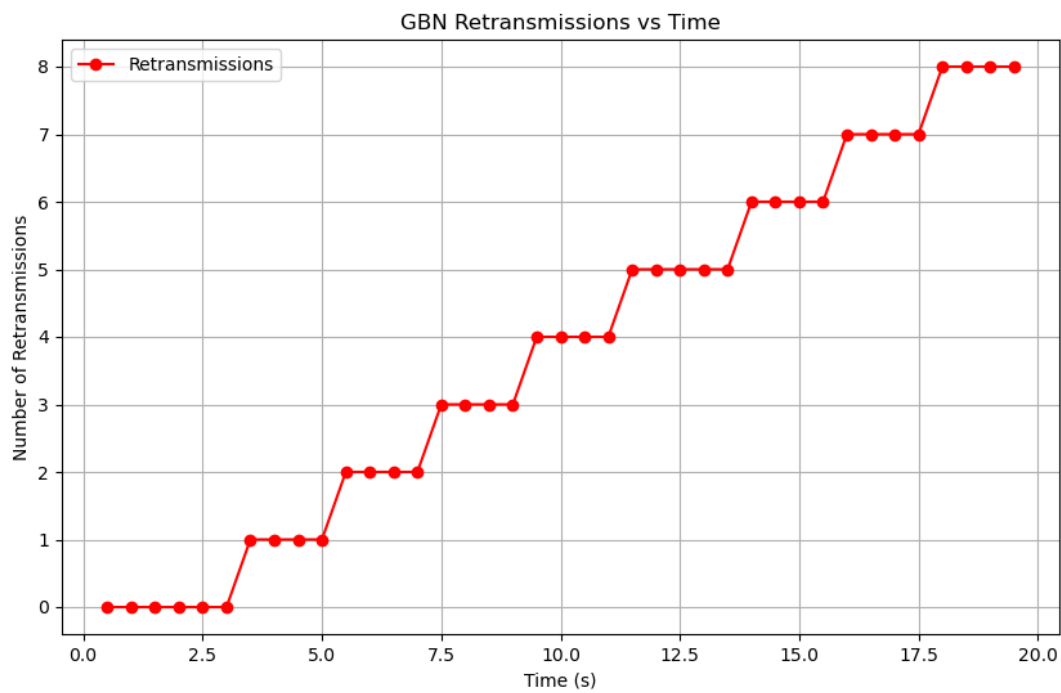
Sender: Total retransmissions = 13
```

1.6.2 GBN Results

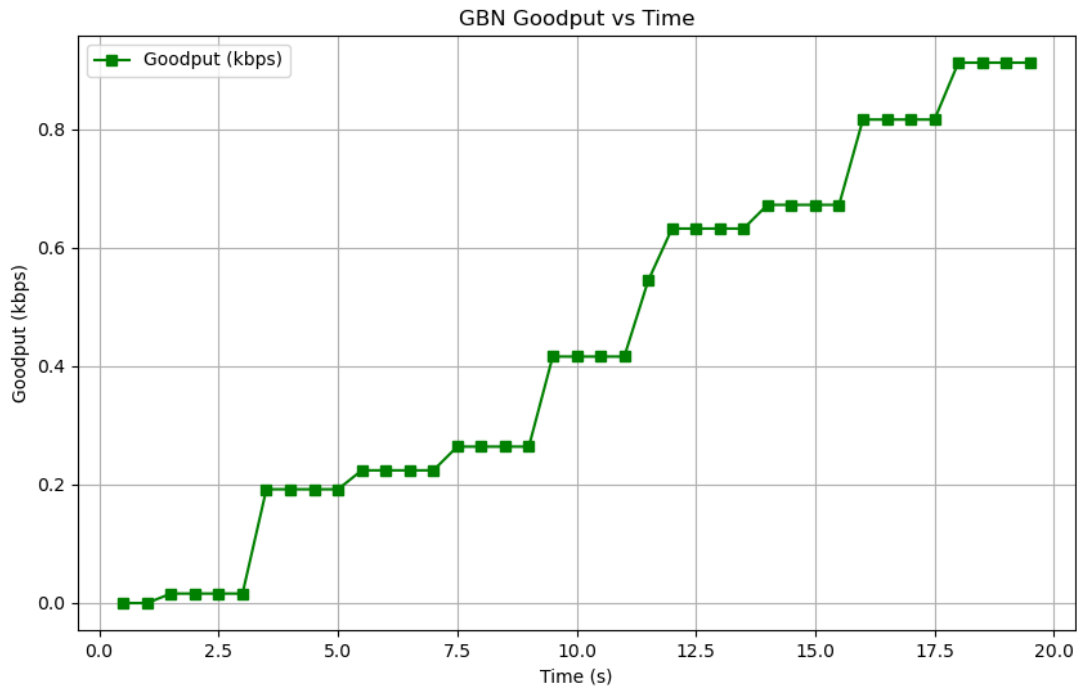
Window Utilization vs. Time:



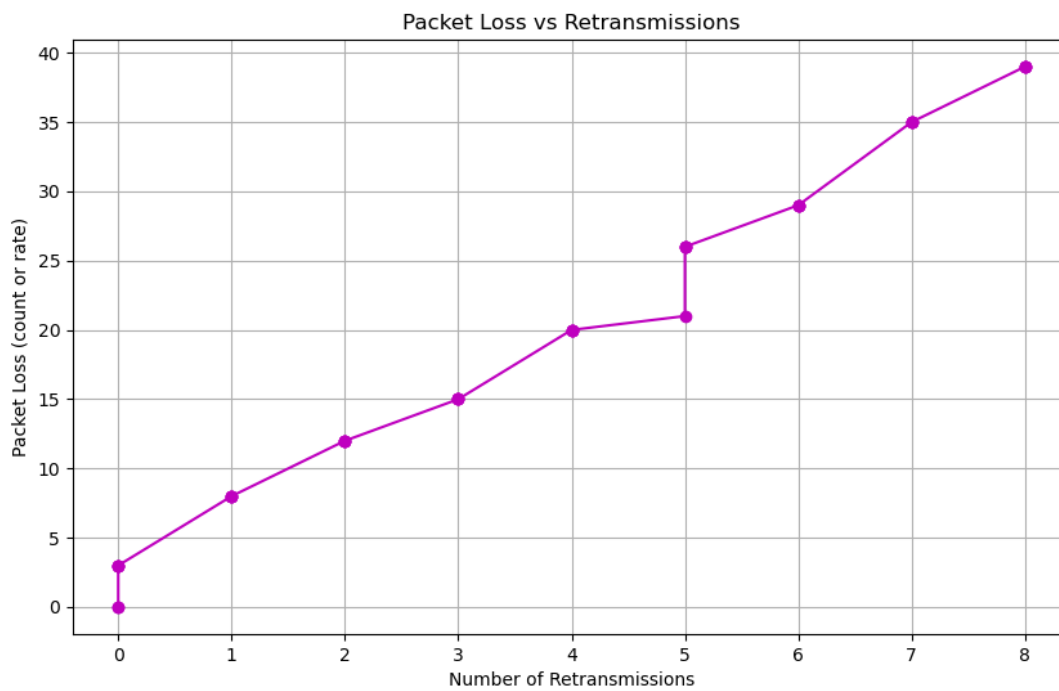
Retransmissions vs. Time:



Goodput vs. Time:



Packet Loss vs. Retransmissions:



```

Sender: Received ACK for Seq=1 at 19.8877
Sender: Sent packet seq=5 at 19.8877
Receiver: Received in-order packet Seq=2 at 19.8972
Receiver: Sent cumulative ACK=2 at 19.8972
Receiver: Received in-order packet Seq=3 at 19.8975
Receiver: Sent cumulative ACK=3 at 19.8975
Receiver: Received in-order packet Seq=4 at 19.8977
Receiver: Sent cumulative ACK=4 at 19.8977
Receiver: Received in-order packet Seq=5 at 19.898
Receiver: Sent cumulative ACK=5 at 19.898
Sender: Received ACK for Seq=2 at 19.9075
Sender: Sent packet seq=6 at 19.9075
Sender: Received ACK for Seq=3 at 19.9077
Sender: Sent packet seq=7 at 19.9077
Sender: Received ACK for Seq=4 at 19.908
Sender: Sent packet seq=0 at 19.908
Sender: Received ACK for Seq=5 at 19.9083
Sender: Sent packet seq=1 at 19.9083

Receiver: Packet lost/corrupted or out-of-order! Seq=6, Expected=6 at 19.9177
Receiver: Received in-order packet Seq=6 at 19.9177
Receiver: Sent cumulative ACK=6 at 19.9177
Receiver: Received in-order packet Seq=7 at 19.918
Receiver: Sent cumulative ACK=7 at 19.918
Receiver: Received in-order packet Seq=0 at 19.9183
Receiver: Sent cumulative ACK=0 at 19.9183
Receiver: Received in-order packet Seq=1 at 19.9186
Receiver: Sent cumulative ACK=1 at 19.9186
Sender: Received ACK for Seq=6 at 19.928
Sender: Sent packet seq=2 at 19.928
Sender: Received ACK for Seq=7 at 19.9283
Sender: Sent packet seq=3 at 19.9283
Sender: Received ACK for Seq=0 at 19.9286
Sender: Sent packet seq=4 at 19.9286
Sender: Received ACK for Seq=1 at 19.9288
Sender: Sent packet seq=5 at 19.9288
Receiver: Received in-order packet Seq=2 at 19.9383
Receiver: Sent cumulative ACK=2 at 19.9383

Receiver: Packet lost/corrupted or out-of-order! Seq=4, Expected=3 at 19.9388
Receiver: Sent cumulative ACK=2 at 19.9388

Receiver: Packet lost/corrupted or out-of-order! Seq=5, Expected=3 at 19.9391
Receiver: Sent cumulative ACK=2 at 19.9391
Sender: Received ACK for Seq=2 at 19.9486
Sender: Sent packet seq=6 at 19.9486
Sender: Received ACK for Seq=2 at 19.9491
Sender: Received ACK for Seq=2 at 19.9494

Receiver: Packet lost/corrupted or out-of-order! Seq=6, Expected=3 at 19.9588
Receiver: Sent cumulative ACK=2 at 19.9588
Sender: Received ACK for Seq=2 at 19.9691

Retransmissions: 9
Lost packets: 45

```

2 Problem 2: TCP Congestion Control

2.1 Introduction

TCP (Transmission Control Protocol) congestion control is a critical mechanism that enables the Internet to function efficiently by preventing network congestion collapse. When multiple flows compete for network resources, congestion control algorithms adjust the sending rate to ensure fair resource allocation and prevent buffer overflow at routers.

This problem explores six different TCP congestion control algorithms—TCP Veno, Hybla, New Reno, HighSpeed, Vegas, and Westwood—analyzing their behavior under competing traffic and evaluating their performance through congestion window dynamics, packet drops, and throughput.

2.2 Theoretical Background

2.2.1 TCP Congestion Control Fundamentals (Problem 2)

Theory

TCP congestion control operates through four key mechanisms:

1. **Slow Start:** Exponentially increases congestion window (cwnd) until threshold
2. **Congestion Avoidance:** Linear increase of cwnd after threshold
3. **Fast Retransmit:** Retransmit lost segment upon receiving 3 duplicate ACKs
4. **Fast Recovery:** Recover from loss without returning to slow start

Congestion Window Dynamics:

The congestion window determines how many bytes can be in flight (unacknowledged) at any time:

$$\text{Sending Rate} = \frac{cwnd}{RTT}$$

Different TCP variants modify these mechanisms to optimize for specific network conditions.

2.2.2 TCP Variants Overview (Problem 2)

Theory

1. TCP New Reno:

- Standard TCP variant, baseline for comparison
- Improves upon TCP Reno by handling multiple packet losses better
- Fast recovery continues until all losses in window are recovered

2. TCP Vegas:

- Proactive congestion avoidance based on RTT measurements
- Adjusts cwnd before packet loss occurs
- Better for low-loss networks but can be aggressive against other variants

3. TCP Veno:

- Distinguishes between congestive and random losses
- Combines Vegas's RTT-based detection with Reno's loss recovery
- Performs well in wireless networks with random losses

4. TCP Westwood:

- Uses bandwidth estimation to set cwnd after loss
- Better performance in wireless and high-loss networks
- Maintains efficiency while being TCP-friendly

5. TCP Hybla:

- Designed for satellite and long-RTT networks
- Eliminates RTT-based performance penalty
- Faster convergence to available bandwidth

6. TCP HighSpeed:

- Designed for high-bandwidth, long-delay networks
- More aggressive window increase in high-speed scenarios
- Improves utilization of high-capacity links

2.3 Network Topology and Setup

2.3.1 Problem 2: TCP Topology

Theory

Network Configuration:

- Two nodes: N0 (sender) and N1 (receiver)
- Point-to-point link: 1 Mbps bandwidth, 10 ms delay
- Queue type: DropTail
- Queue size: Based on bandwidth-delay product (BDP)

Bandwidth-Delay Product Calculation:

$$BDP = Bandwidth \times RTT = 1 \text{ Mbps} \times 20 \text{ ms} = 2500 \text{ bytes}$$

Traffic Configuration:

- 1 FTP flow (TCP-based, bulk transfer)
- 5 CBR flows (UDP-based, 500 Kbps each)
- Total competing traffic: 2.5 Mbps CBR + TCP FTP

Flow Timing Schedule:

Flow	Start Time (ms)	Stop Time (ms)
FTP	0	2000
CBR1	300	2000
CBR2	500	2000
CBR3	700	1100
CBR4	800	1500
CBR5	1000	1600

Table 1: Flow Timing Schedule for TCP Congestion Control Experiment

2.4 Implementation

2.5 Results and Analysis

2.5.1 Problem 2: Task 1 - Congestion Window Dynamics

TCP New Reno:

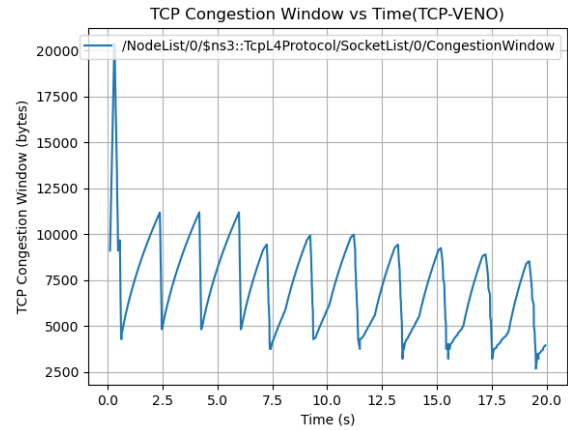
Page 20 of 34

```

18.7984s: cwnd changed 7276 -> 7315 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.7998s: cwnd changed 7315 -> 7354 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.8092s: cwnd changed 7354 -> 7393 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.8187s: cwnd changed 7393 -> 7431 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.8281s: cwnd changed 7431 -> 7469 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.8376s: cwnd changed 7469 -> 7507 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.847s: cwnd changed 7507 -> 7545 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.8564s: cwnd changed 7545 -> 7583 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.8659s: cwnd changed 7583 -> 7620 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.8753s: cwnd changed 7620 -> 7657 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.8848s: cwnd changed 7657 -> 7694 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.8942s: cwnd changed 7694 -> 7731 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9036s: cwnd changed 7731 -> 7768 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9131s: cwnd changed 7768 -> 7804 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9225s: cwnd changed 7804 -> 7840 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.932s: cwnd changed 7840 -> 7876 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9414s: cwnd changed 7876 -> 7912 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9508s: cwnd changed 7912 -> 7948 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9603s: cwnd changed 7948 -> 7984 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9697s: cwnd changed 7984 -> 8019 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9792s: cwnd changed 8019 -> 8054 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9886s: cwnd changed 8054 -> 8089 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.998s: cwnd changed 8089 -> 8124 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.0075s: cwnd changed 8124 -> 8159 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.0169s: cwnd changed 8159 -> 8194 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.0264s: cwnd changed 8194 -> 8229 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.0358s: cwnd changed 8229 -> 8263 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.0452s: cwnd changed 8263 -> 8297 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.0547s: cwnd changed 8297 -> 8331 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.0641s: cwnd changed 8331 -> 8365 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.0736s: cwnd changed 8365 -> 8399 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.083s: cwnd changed 8399 -> 8433 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.0925s: cwnd changed 8433 -> 8467 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.1019s: cwnd changed 8467 -> 8500 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.1113s: cwnd changed 8500 -> 8533 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.1208s: cwnd changed 8533 -> 8566 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.1302s: cwnd changed 8566 -> 8599 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.1397s: cwnd changed 8599 -> 8631 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.1491s: cwnd changed 8631 -> 8664 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.1586s: cwnd changed 8664 -> 8696 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.168s: cwnd changed 8696 -> 8728 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.1775s: cwnd changed 8728 -> 8759 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.1869s: cwnd changed 8759 -> 8790 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.1964s: cwnd changed 8790 -> 8821 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2058s: cwnd changed 8821 -> 8851 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2153s: cwnd changed 8851 -> 8881 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2247s: cwnd changed 8881 -> 8911 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2342s: cwnd changed 8911 -> 8941 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2436s: cwnd changed 8941 -> 8970 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2531s: cwnd changed 8970 -> 9000 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2625s: cwnd changed 9000 -> 9029 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.272s: cwnd changed 9029 -> 9058 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2815s: cwnd changed 9058 -> 9087 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2909s: cwnd changed 9087 -> 9116 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3004s: cwnd changed 9116 -> 9145 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3098s: cwnd changed 9145 -> 9174 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3193s: cwnd changed 9174 -> 9203 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3287s: cwnd changed 9203 -> 9232 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3382s: cwnd changed 9232 -> 9261 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3476s: cwnd changed 9261 -> 9290 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3571s: cwnd changed 9290 -> 9319 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3665s: cwnd changed 9319 -> 9348 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.376s: cwnd changed 9348 -> 9377 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3855s: cwnd changed 9377 -> 9406 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3949s: cwnd changed 9406 -> 9435 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4044s: cwnd changed 9435 -> 9464 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4138s: cwnd changed 9464 -> 9493 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4233s: cwnd changed 9493 -> 9522 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4327s: cwnd changed 9522 -> 9551 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4422s: cwnd changed 9551 -> 9580 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4516s: cwnd changed 9580 -> 9609 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4611s: cwnd changed 9609 -> 9638 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4705s: cwnd changed 9638 -> 9667 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.48s: cwnd changed 9667 -> 9696 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4895s: cwnd changed 9696 -> 9724 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4989s: cwnd changed 9724 -> 9753 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5084s: cwnd changed 9753 -> 9782 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5178s: cwnd changed 9782 -> 9811 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5273s: cwnd changed 9811 -> 9840 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5367s: cwnd changed 9840 -> 9869 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5462s: cwnd changed 9869 -> 9898 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5556s: cwnd changed 9898 -> 9927 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5651s: cwnd changed 9927 -> 9956 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5745s: cwnd changed 9956 -> 9985 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.584s: cwnd changed 9985 -> 10014 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5935s: cwnd changed 10014 -> 10043 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6029s: cwnd changed 10043 -> 10072 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6124s: cwnd changed 10072 -> 10101 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6218s: cwnd changed 10101 -> 10130 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6313s: cwnd changed 10130 -> 10159 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6407s: cwnd changed 10159 -> 10188 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6502s: cwnd changed 10188 -> 10217 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6596s: cwnd changed 10217 -> 10246 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6691s: cwnd changed 10246 -> 10275 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6785s: cwnd changed 10275 -> 10304 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.688s: cwnd changed 10304 -> 10333 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6974s: cwnd changed 10333 -> 10362 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.7069s: cwnd changed 10362 -> 10391 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.7163s: cwnd changed 10391 -> 10420 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.7258s: cwnd changed 10420 -> 10449 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.7352s: cwnd changed 10449 -> 10478 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.7447s: cwnd changed 10478 -> 10507 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.7541s: cwnd changed 10507 -> 10536 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.7636s: cwnd changed 10536 -> 10565 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.773s: cwnd changed 10565 -> 10594 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.7825s: cwnd changed 10594 -> 10623 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.7919s: cwnd changed 10623 -> 10652 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.8014s: cwnd changed 10652 -> 10681 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.8108s: cwnd changed 10681 -> 10710 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.8203s: cwnd changed 10710 -> 10739 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.8297s: cwnd changed 10739 -> 10768 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.8392s: cwnd changed 10768 -> 10797 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.8486s: cwnd changed 10797 -> 10826 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.8581s: cwnd changed 10826 -> 10855 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.8675s: cwnd changed 10855 -> 10884 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.877s: cwnd changed 10884 -> 10913 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.8864s: cwnd changed 10913 -> 10942 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.8959s: cwnd changed 10942 -> 10971 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.9053s: cwnd changed 10971 -> 11000 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.9148s: cwnd changed 11000 -> 11029 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.9242s: cwnd changed 11029 -> 11058 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.9337s: cwnd changed 11058 -> 11087 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.9431s: cwnd changed 11087 -> 11116 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.9526s: cwnd changed 11116 -> 11145 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.962s: cwnd changed 11145 -> 11174 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.9715s: cwnd changed 11174 -> 11203 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.9809s: cwnd changed 11203 -> 11232 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.9904s: cwnd changed 11232 -> 11261 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
20.0s: TCP Variant: ns3::TcpVeno
Cumulative packets dropped: 439
Cumulative bytes received: 2581432

```

(a) Output Graph



(b) Congestion Window

Figure 3: TCP Veno Performance Analysis

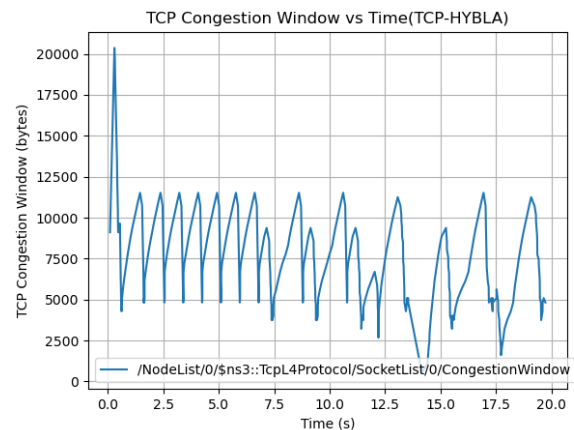
TCP Hybla:

```

17.0724s: cwnd changed 7889 -> 7574 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.0772s: cwnd changed 7574 -> 7260 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.0819s: cwnd changed 7260 -> 6945 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.1288s: cwnd changed 6945 -> 6695 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.1672s: cwnd changed 6695 -> 5780 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.172s: cwnd changed 5780 -> 5892 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.1767s: cwnd changed 5892 -> 5992 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.292s: cwnd changed 5992 -> 5892 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.2967s: cwnd changed 5892 -> 4824 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.3483s: cwnd changed 4824 -> 4288 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.353s: cwnd changed 4288 -> 5892 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.3577s: cwnd changed 5892 -> 4824 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.3599s: cwnd changed 4824 -> 5892 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.5209s: cwnd changed 5892 -> 5628 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.5941s: cwnd changed 5628 -> 4824 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.5968s: cwnd changed 4824 -> 4288 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.6458s: cwnd changed 4288 -> 3752 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.6585s: cwnd changed 3752 -> 3395 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.7143s: cwnd changed 3395 -> 2144 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.7238s: cwnd changed 2144 -> 1680 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.7766s: cwnd changed 1680 -> 2144 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.8385s: cwnd changed 2144 -> 2680 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.8821s: cwnd changed 2680 -> 3216 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.0032s: cwnd changed 3216 -> 3752 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.1664s: cwnd changed 3752 -> 4288 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.2716s: cwnd changed 4288 -> 4824 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.3094s: cwnd changed 4824 -> 5360 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.3566s: cwnd changed 5360 -> 5896 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.4132s: cwnd changed 5896 -> 6432 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.4698s: cwnd changed 6432 -> 6968 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.5265s: cwnd changed 6968 -> 7504 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.5926s: cwnd changed 7504 -> 8040 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.6866s: cwnd changed 8040 -> 8576 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.7342s: cwnd changed 8576 -> 9112 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.8191s: cwnd changed 9112 -> 9648 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.941s: cwnd changed 9648 -> 10184 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9891s: cwnd changed 10184 -> 10720 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.0834s: cwnd changed 10720 -> 11256 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2346s: cwnd changed 11256 -> 10720 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2894s: cwnd changed 10720 -> 10184 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3831s: cwnd changed 10184 -> 9777 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.381s: cwnd changed 9777 -> 8926 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3848s: cwnd changed 8926 -> 8611 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3959s: cwnd changed 8611 -> 7761 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4232s: cwnd changed 7761 -> 7446 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.428s: cwnd changed 7446 -> 7132 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4644s: cwnd changed 7132 -> 6281 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4711s: cwnd changed 6281 -> 5966 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4759s: cwnd changed 5966 -> 5892 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5227s: cwnd changed 5892 -> 4824 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5275s: cwnd changed 4824 -> 4288 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5322s: cwnd changed 4288 -> 3752 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5959s: cwnd changed 3752 -> 4288 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6866s: cwnd changed 4288 -> 4824 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6475s: cwnd changed 4824 -> 5892 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.7254s: cwnd changed 5892 -> 4824 [NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.73s: TCP Variant: ns3::TcpHybla
Cumulative packets dropped: 446
Cumulative bytes received: 2550248

```

(a) Output Graph



(b) Congestion Window

Figure 4: TCP Hybla Performance Analysis

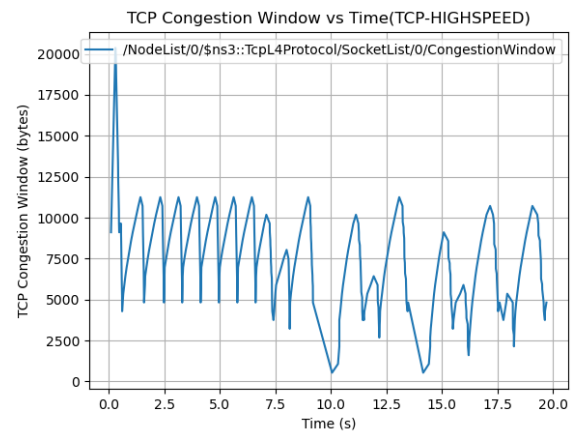
TCP HighSpeed:


```

16.8834s: cwnd changed 8576 -> 9112 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
16.9283s: cwnd changed 9112 -> 9648 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.0133s: cwnd changed 9648 -> 10184 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.1741s: cwnd changed 10184 -> 10720 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.3037s: cwnd changed 10720 -> 10184 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.3596s: cwnd changed 10184 -> 9648 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.3554s: cwnd changed 9648 -> 9234 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.3691s: cwnd changed 9234 -> 8382 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.4339s: cwnd changed 8382 -> 8065 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.4286s: cwnd changed 8065 -> 7748 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.4333s: cwnd changed 7748 -> 6895 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.4705s: cwnd changed 6895 -> 6379 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.4765s: cwnd changed 6379 -> 6262 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.5234s: cwnd changed 6262 -> 4824 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.5328s: cwnd changed 4824 -> 4288 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.6033s: cwnd changed 4288 -> 4824 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.7728s: cwnd changed 4824 -> 3752 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.8968s: cwnd changed 3752 -> 4824 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
17.9454s: cwnd changed 4824 -> 5368 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.1951s: cwnd changed 5368 -> 4824 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.1998s: cwnd changed 4824 -> 4288 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.2868s: cwnd changed 4288 -> 3216 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.2431s: cwnd changed 3216 -> 2859 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.2477s: cwnd changed 2859 -> 2144 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.2881s: cwnd changed 2144 -> 3752 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.3133s: cwnd changed 3752 -> 4288 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.3511s: cwnd changed 4288 -> 4824 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.3982s: cwnd changed 4824 -> 5368 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.4545s: cwnd changed 5368 -> 5996 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.4926s: cwnd changed 5996 -> 6432 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.5493s: cwnd changed 6432 -> 6968 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.6154s: cwnd changed 6968 -> 7504 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.6814s: cwnd changed 7504 -> 8040 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.7475s: cwnd changed 8040 -> 8576 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.8231s: cwnd changed 8576 -> 9112 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9088s: cwnd changed 9112 -> 9648 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
18.9931s: cwnd changed 9648 -> 10184 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.0779s: cwnd changed 10184 -> 10720 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.2975s: cwnd changed 10720 -> 10184 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3444s: cwnd changed 10184 -> 9648 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3491s: cwnd changed 9648 -> 9234 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.3539s: cwnd changed 9234 -> 8918 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4177s: cwnd changed 8918 -> 8601 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4224s: cwnd changed 8601 -> 8284 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4699s: cwnd changed 8284 -> 7431 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4656s: cwnd changed 7431 -> 7115 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.4783s: cwnd changed 7115 -> 6262 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5172s: cwnd changed 6262 -> 5945 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5219s: cwnd changed 5945 -> 5628 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5875s: cwnd changed 5628 -> 4824 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.5951s: cwnd changed 4824 -> 4288 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6425s: cwnd changed 4288 -> 3752 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.6467s: cwnd changed 3752 -> 4288 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
19.7105s: cwnd changed 4288 -> 4824 [/NodeList/0/$ns3:Tcp4Protocol/SocketList/0/CongestionWindow]
-- TCP Variant: ns3:TcpHighSpeed --
Cumulative packets dropped: 440
Cumulative bytes received: 2514288

```

(a) Output Graph



(b) Congestion Window

Figure 5: TCP HighSpeed Performance Analysis

TCP Westwood: not compatible for the current ns3

2.5.2 Problem 2: Task 2 - Cumulative Packet Drops

TCP New Reno:

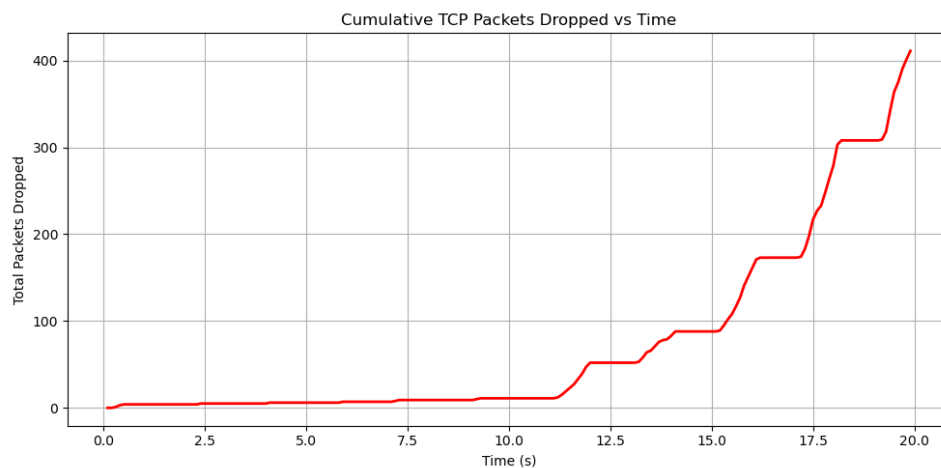


Figure 6: Cumulative Packets Dropped vs. Time for TCP New Reno

TCP Vegas:

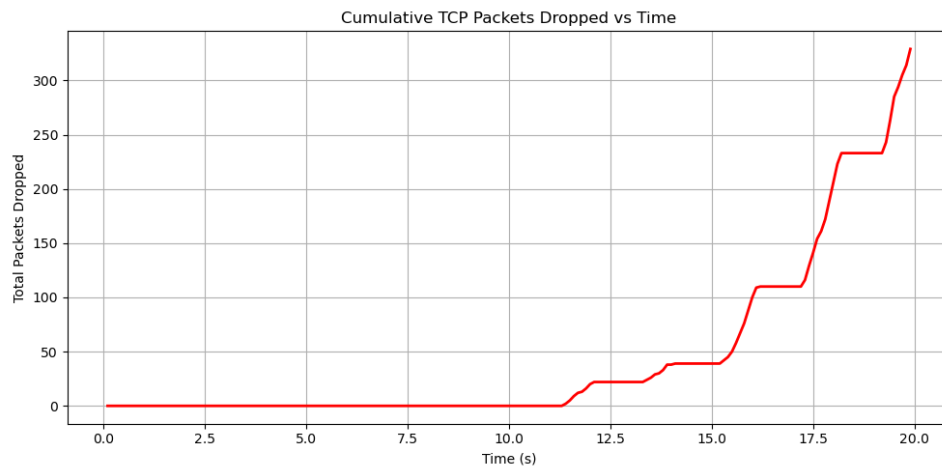


Figure 7: Cumulative Packets Dropped vs. Time for TCP Vegas

TCP Veno:

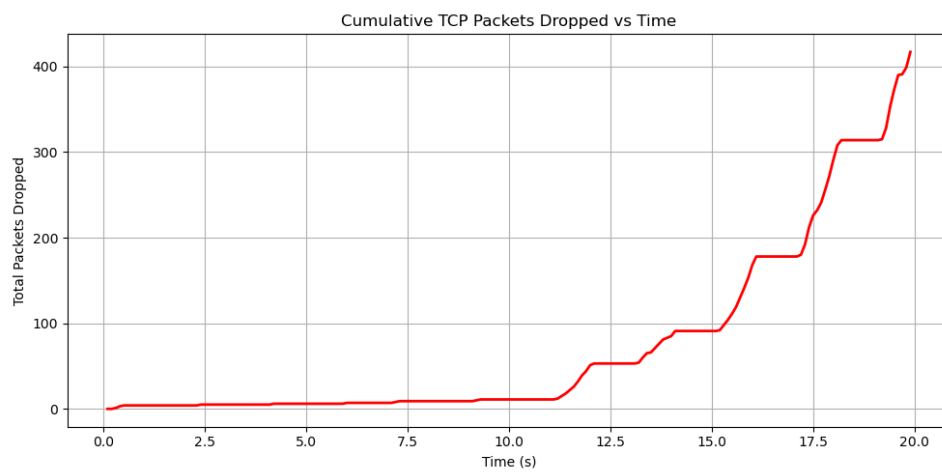


Figure 8: Cumulative Packets Dropped vs. Time for TCP Veno

TCP Hybla:

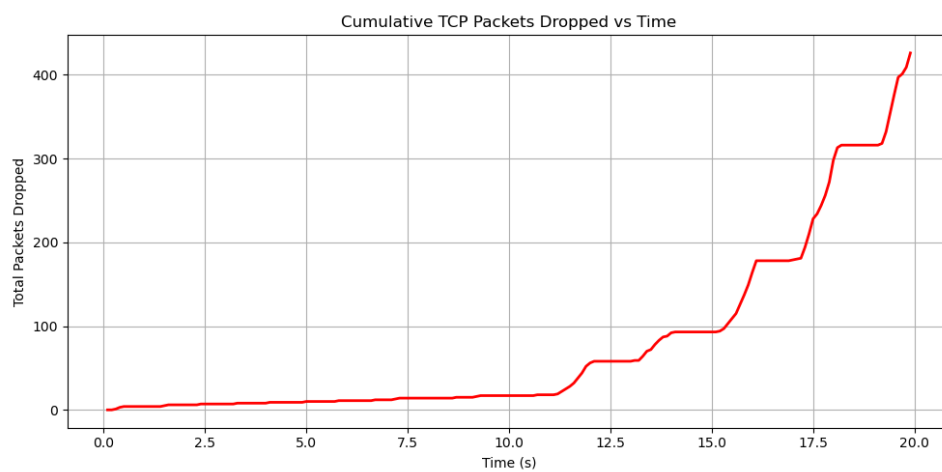


Figure 9: Cumulative Packets Dropped vs. Time for TCP Hybla

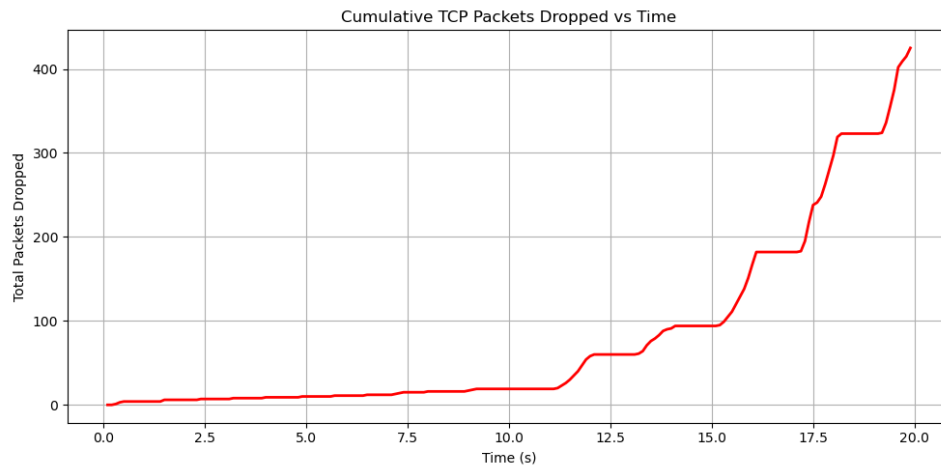
TCP HighSpeed:

Figure 10: Cumulative Packets Dropped vs. Time for TCP HighSpeed

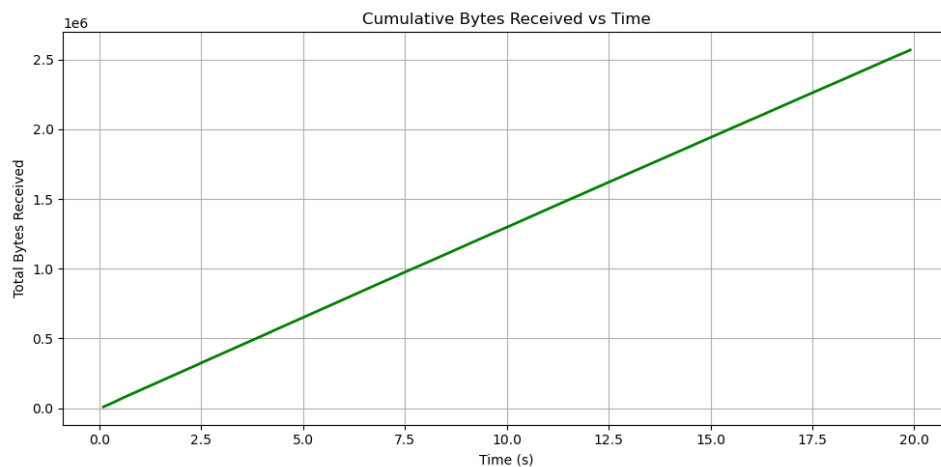
2.5.3 Problem 2: Task 3 - Cumulative Bytes Transferred**TCP New Reno:**

Figure 11: Cumulative Bytes Transferred vs. Time for TCP New Reno

TCP Vegas:

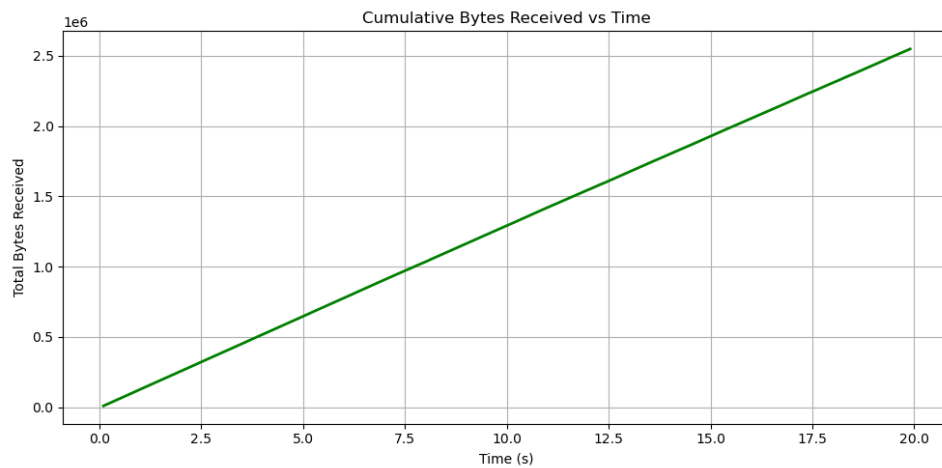


Figure 12: Cumulative Bytes Transferred vs. Time for TCP Vegas

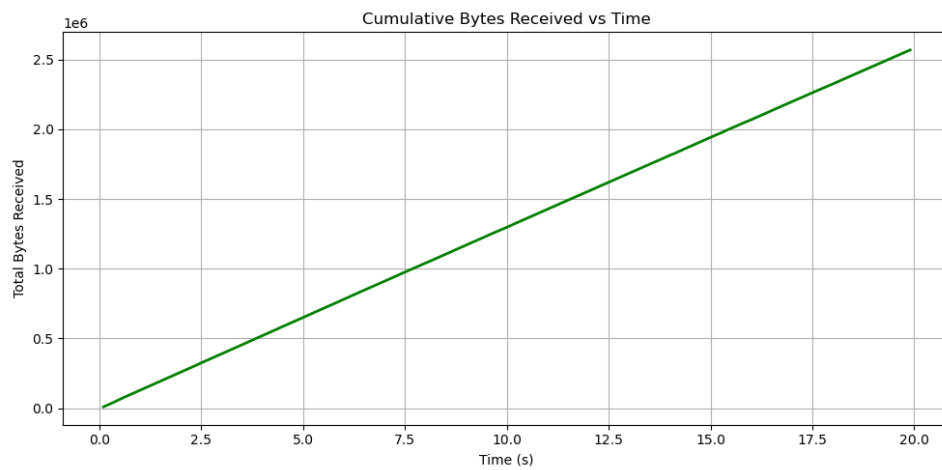
TCP Veno:

Figure 13: Cumulative Bytes Transferred vs. Time for TCP Veno

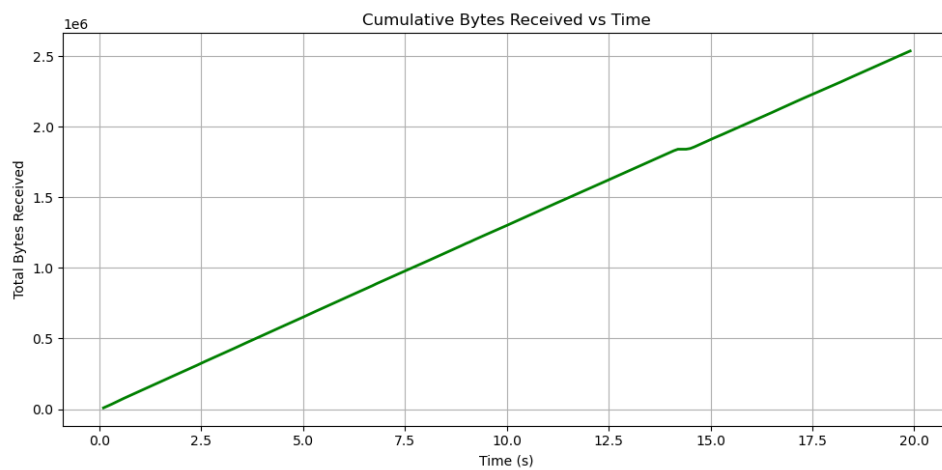
TCP Hybla:

Figure 14: Cumulative Bytes Transferred vs. Time for TCP Hybla

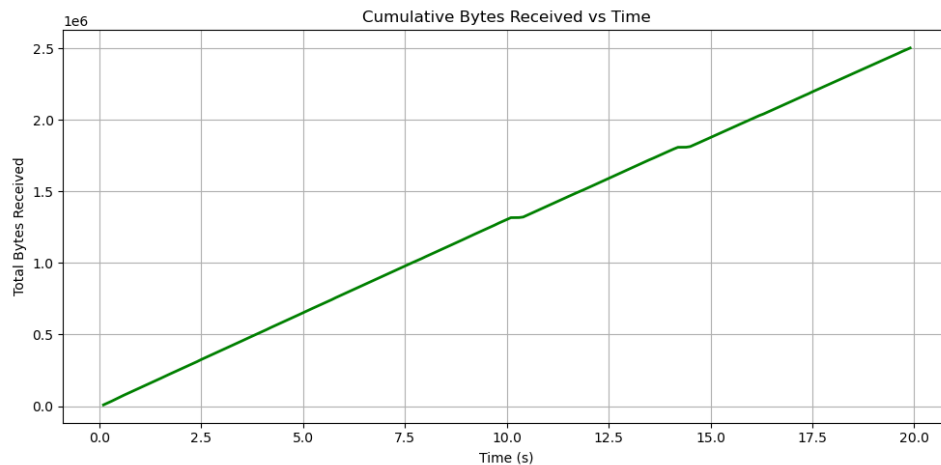
TCP HighSpeed:

Figure 15: Cumulative Bytes Transferred vs. Time for TCP HighSpeed

3 Problem 3: Distance Vector Routing Algorithm and Count-to-Infinity Problem

3.1 Introduction

Distance Vector Routing is a distributed routing algorithm where routers exchange information with their neighbors to build routing tables. Each router maintains a distance vector containing the best-known cost to reach every destination in the network. The algorithm is based on the Bellman-Ford equation and operates iteratively until routing tables converge.

However, Distance Vector Routing suffers from the well-known Count-to-Infinity problem when link failures occur. When a link fails, incorrect routing information can propagate through the network, causing routers to continuously increase their distance estimates to unreachable destinations. This problem demonstrates fundamental challenges in distributed systems and the importance of mitigation techniques.

3.2 Theoretical Background

Theory

3.2.1 Bellman-Ford Equation

The core of Distance Vector Routing is the Bellman-Ford equation:

$$D_x(y) = \min_{v \in N(x)} \{c(x, v) + D_v(y)\}$$

3.2.2 Algorithm Operation

Initialization:

1. Each router initializes its distance to directly connected neighbors as the link cost
2. Distance to all other nodes is initialized to infinity (∞)
3. Each router knows only its local link costs

Iterative Updates:

1. Each router periodically sends its distance vector to all neighbors
2. Upon receiving a distance vector from neighbor v , router x updates:

$$D_x(y) \leftarrow \min\{D_x(y), c(x, v) + D_v(y)\}$$

3. If any distance changes, router sends updated vector to neighbors
4. Process continues until no updates occur (convergence)

3.2.3 Count-to-Infinity Problem

When a link fails or its cost increases dramatically:

1. Router detects failure and updates its distance to ∞
2. Neighboring routers may have routes through the failed router
3. These neighbors don't immediately realize their routes are invalid
4. Incorrect information propagates, causing distance estimates to increase incrementally
5. Routers "count to infinity" before realizing destination is unreachable

3.3 Network Topology

Link Costs:

- A—B: 1
- A—C: 3
- B—D: 1
- C—D: 2

3.4 Mitigation Techniques

3.4.1 Split Horizon

Theory

Principle: Never advertise a route back to the neighbor from which it was learned.

Logic:

- If router x uses neighbor v to reach destination y
- Then x should not advertise route to y back to v
- Prevents simple routing loops

Limitation: Does not prevent loops involving three or more routers.

3.4.2 Poison Reverse

Theory

Principle: Advertise infinite distance for routes learned from a neighbor back to that neighbor.

Logic:

- If router x routes to y through neighbor v
- x advertises distance ∞ to y when sending to v
- Explicitly tells v not to route through x for destination y

Advantage: More explicit than split horizon, breaks loops faster.

Limitation: Still doesn't solve all Count-to-Infinity scenarios (e.g., loops involving 3+ nodes).

3.5 Implementation Code

```

}
for(int q=0; q<V; q++){
    vector<vector<int>> newDist = dist;
    for(int i=0; i<V; i++){
        for(int j=0; j<adj[i].size(); j++){
            int node=adj[i][j].first;
            int cost=adj[i][j].second;
            for(int k=0; k<V; k++){
                //k is the destination
                //dist[node][k]=min(dist[node][k],cost+dist[i][k]);
                if(dist[i][k]>cost+dist[node][k]){
                    newDist[i][k]=cost+dist[node][k];
                }
            }
        }
    }
    dist=newDist;
    cout << "TABLE IN ITERATION " << q << "\n";
    for(int i=0; i<V; i++){
        for(int j=0; j<V; j++){
            cout << dist[i][j] << " ";
        }
        cout << "\n";
    }
}

```

```

for(int tc=0; tc<10; tc++){
    vector<vector<int>> newDist=dist;
    for(int i=0; i<V; i++){
        //node i receives
        // int minm=INF+1000;
        vector<int> minm(V,INF+1000);
        for(int j=0; j<adj[i].size(); j++){
            int node=adj[i][j].first;
            int cost=adj[i][j].second;
            if(cost!=INF){
                //then only it will relay its TABLE
                for(int k=0; k<V; k++){
                    minm[k]=min(minm[k],cost+dist[node][k]);
                    if(i!=k){
                        newDist[i][k]=minm[k];
                    }
                }
            }
        }
    }
    dist=newDist;
    cout << "TABLE IN ITEARTION " << tc << "\n";
    printtable(V,dist);
}

```

```

for(int tc=0; tc<10; tc++){
    vector<vector<int>> newDist=dist;
    vector<vector<int>> newNext=nextHop;
    for(int i=0; i<V; i++){
        // node i receives from neighbors, but will ignore (split horizon) routes that neighbor learned via i
        vector<int> minm(V,INF+1000); // temporary minima per destination for node i
        for(int j=0; j<adj[i].size(); j++){

            int node=adj[i][j].first;
            int cost=adj[i][j].second;
            if(cost==INF) continue;
            // neighbor node will advertise its vector but skip entries learned via i (split horizon)
            for(int k=0; k<V; k++){
                if(k==i) continue;
                if(dist[node][k]==INF) continue;
                if(nextHop[node][k]==i) continue; // SPLIT HORIZON: skip route
                long long cand = (long long)cost + dist[node][k];
                if(cand >= INF) continue;
                if(cand < minm[k]) minm[k] = (int)cand;
            }
        }
        // copy minima into newDist/newNext (set nextHop to the neighbor that gave the min)
        for(int k=0; k<V; k++){
            if(k==i) continue;
            if(minm[k] < newDist[i][k]){
                // we need to find which neighbor provided this minm[k]
                int chosenNeighbor = -1;
                for(int j=0; j<adj[i].size(); j++){
                    int node = adj[i][j].first;
                    int cost = adj[i][j].second;
                    if(cost==INF) continue;
                    if(dist[node][k]==INF) continue;
                    if(nextHop[node][k]==i) continue; // split horizon
                    long long cand = (long long)cost + dist[node][k];
                    if(cand == minm[k]){
                        chosenNeighbor = node;
                        break;
                    }
                }
                if(chosenNeighbor!=-1){
                    newDist[i][k] = minm[k];
                    newNext[i][k] = chosenNeighbor;
                }
            }
        }
    }
    dist=newDist;
    nextHop=newNext;
    cout << "TABLE IN ITERATION " << tc << "\n";
    printtable(V,dist);
}

```

```

cout << "POISON REVERSE SIMULATION\n\n\n";
int x,y;
cin >> x >> y;

for(int i=0; i<adj[x].size(); i++){
    if(adj[x][i].first==y) adj[x][i].second=INF;
}
for(int i=0; i<adj[y].size(); i++){
    if(adj[y][i].first==x) adj[y][i].second=INF;
}

cout << "INITIAL TABLE AFTER LINK DESTRUCTION\n\n";
printtable(V,dist);

for(int tc=0; tc<10; tc++){
    vector<vector<int>> newDist=dist;

    for(int i=0; i<V; i++){
        vector<int> minm(V,INF+1000);
        for(int j=0; j<adj[i].size(); j++){
            int node=adj[i][j].first;
            int cost=adj[i][j].second;

            if(cost!=INF){
                for(int k=0; k<V; k++){
                    int advertiseCost = dist[node][k];

                    // Poison Reverse:
                    // If node j's best route to k is through i,
                    // then j will advertise distance to k as INF to i.
                    if(dist[node][k] == dist[node][i] + adj[node][i].second)
                        advertiseCost = INF;

                    minm[k] = min(minm[k], cost + advertiseCost);
                    if(i!=k) newDist[i][k] = minm[k];
                }
            }
        }
    }

    dist = newDist;
    cout << "TABLE IN ITERATION " << tc << "\n";
    printtable(V,dist);
}

```

3.6 Simulation Results

3.6.1 Initial Convergence (Before Link Failure)

Routing Tables at Convergence:

INITIAL TABLE

0 1 3 100000000

1 0 1 1

3 1 0 2

100000000 1 2 0

TABLE IN ITERATION 0

0 1 2 5

1 0 1 1

2 1 0 2

5 1 2 0

TABLE IN ITERATION 1

0 1 2 2

1 0 1 1

2 1 0 2

4 1 2 0

TABLE IN ITERATION 2

0 1 2 2

1 0 1 1

2 1 0 2

2 1 2 0

TABLE IN ITERATION 3

0 1 2 2

1 0 1 1

2 1 0 2

2 1 2 0

1

INITIAL TABLE AFTER LINK DESTRUCTION

0 1 2 2

1 0 1 1

2 1 0 2

2 1 2 0

TABLE IN ITEARTION 0

0 1 2 2

1 0 1 3

2 1 0 2

4 3 2 0

TABLE IN ITEARTION 1

0 1 2 4

1 0 1 3

2 1 0 2

4 3 2 0

TABLE IN ITEARTION 2

0 1 2 4

1 0 1 3

2 1 0 2

4 3 2 0

TABLE IN ITEARTION 3

0 1 2 4

1 0 1 3

2 1 0 2

4 3 2 0

TABLE IN ITEARTION 4

0 1 2 4

1 0 1 3

2 1 0 2

4 3 2 0

TABLE IN ITEARTION 5

0 1 2 4

1 0 1 3

2 1 0 2

4 3 2 0

TABLE IN ITEARTION 6

0 1 2 4

1 0 1 3

2 1 0 2

4 3 2 0

TABLE IN ITEARTION 7

0 1 2 4

1 0 1 3

2 1 0 2

4 3 2 0

TABLE IN ITEARTION 8

0 1 2 4

1 0 1 3

2 1 0 2

4 3 2 0

3.6.2 Mitigation Results: Split Horizon

```
TABLE IN ITEARTION 0
0 1 2 2
1 0 1 3
2 1 0 2
2 3 2 0
TABLE IN ITEARTION 1
0 1 2 2
1 0 1 3
2 1 0 2
2 3 2 0
TABLE IN ITEARTION 2
0 1 2 2
1 0 1 3
2 1 0 2
2 3 2 0
TABLE IN ITEARTION 3
0 1 2 2
1 0 1 3
2 1 0 2
2 3 2 0
TABLE IN ITEARTION 4
0 1 2 2
1 0 1 3
2 1 0 2
2 3 2 0
TABLE IN ITEARTION 5
0 1 2 2
1 0 1 3
2 1 0 2
2 3 2 0
TABLE IN ITEARTION 6
0 1 2 2
1 0 1 3
2 1 0 2
2 3 2 0
TABLE IN ITEARTION 7
0 1 2 2
1 0 1 3
2 1 0 2
2 3 2 0
TABLE IN ITEARTION 8
0 1 2 2
1 0 1 3
2 1 0 2
2 3 2 0
```

3.6.3 Mitigation Results: Poison Reverse

```
TABLE IN ITERATION 0
0 1 2 2
1 0 1 3
2 1 0 2
1000000002 3 2 0
TABLE IN ITERATION 1
0 1 2 4
1 0 1 3
2 1 0 2
1000000002 3 2 0
TABLE IN ITERATION 2
0 1 2 4
1 0 1 1000000001
2 1 0 2
1000000002 3 2 0
TABLE IN ITERATION 3
0 1 2 5
1 0 1 1000000001
2 1 0 2
1000000002 3 2 0
TABLE IN ITERATION 4
0 1 2 5
1 0 1 6
2 1 0 2
1000000002 3 2 0
TABLE IN ITERATION 5
0 1 2 5
1 0 1 6
2 1 0 2
1000000002 3 2 0
TABLE IN ITERATION 6
0 1 2 5
1 0 1 6
2 1 0 2
1000000002 3 2 0
TABLE IN ITERATION 7
0 1 2 5
1 0 1 6
2 1 0 2
1000000002 3 2 0
TABLE IN ITERATION 8
0 1 2 5
1 0 1 6
2 1 0 2
1000000002 3 2 0
```